Batch 19 — Delivery + Dispatch Deepening Pack

Daniel Macharia <danielmacharia43@gmail.com>
To: <daniel@ics.ac.ke>

Tue, 16 Jun at 08:22

Batch 19 — Delivery + Dispatch Deepening Pack

Meat Lovers CIMS powered by YohPal

19A.

database/migrations/031_delivery_deepening.sql

```
ALTER TABLE deliveries
ADD COLUMN delivery_fee DECIMAL(12,2) DEFAULT 0 AFTER delivery_address,
ADD COLUMN assigned_by BIGINT NULL AFTER rider_phone,
ADD COLUMN dispatched_at TIMESTAMP NULL AFTER assigned_by,
ADD COLUMN delivered_at TIMESTAMP NULL AFTER dispatched_at,
ADD COLUMN failed_at TIMESTAMP NULL AFTER delivered_at,
ADD COLUMN failed_reason TEXT AFTER failed_at,
ADD COLUMN delivery_notes TEXT AFTER failed_reason;

ALTER TABLE deliveries
ADD CONSTRAINT fk_deliveries_assigned_by
FOREIGN KEY (assigned_by) REFERENCES users(id);
```

19B.

database/migrations/032_riders.sql

```
CREATE TABLE riders (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,

    rider_name VARCHAR(255) NOT NULL,
    rider_phone VARCHAR(50) NOT NULL,

    rider_status ENUM(
        'ACTIVE',
        'INACTIVE',
        'SUSPENDED'
    ) DEFAULT 'ACTIVE',

    notes TEXT,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

19C.

database/seeds/016_riders_seed.sql

```
INSERT INTO riders (
    rider_name,
    rider_phone,
    rider_status,
    notes
) VALUES
('Default Rider One', '0733000001', 'ACTIVE', 'Seed rider for delivery testing'),
('Default Rider Two', '0733000002', 'ACTIVE', 'Seed rider for delivery testing');
```

19D. Update

backend/routes/api.php

Add routes:

```
$router->get('/api/deliveries', [DispatchController::class, 'orders']);
```

```
$router->post('/api/deliveries', [DispatchController::class, 'createDeliveryOrder']);

$router->get('/api/riders', [DispatchController::class, 'riders']);
$router->post('/api/riders', [DispatchController::class, 'storeRider']);

$router->post('/api/dispatch/orders/{id}/assign-rider', [DispatchController::class, 'assignRider']);
$router->post('/api/dispatch/orders/{id}/fail', [DispatchController::class, 'failDelivery']);

$router->get('/api/dispatch/performance-report', [DispatchController::class, 'performanceReport']);
```

19E. Update

backend/app/Controllers/DispatchController.php

Add these methods inside the class:

```
public function createDeliveryOrder(): void
{
    $user = Auth::requireRole([
        'SUPER_ADMIN',
        'ADMIN',
        'MANAGER',
        'DISPATCHER',
        'CASHIER'
    ]);

    $data = Request::body();
    $db = Database::connection();

    $stmt = $db->prepare(
        'INSERT INTO deliveries
        (order_id, customer_id, delivery_status, rider_name, rider_phone, delivery_address,
         delivery_fee, assigned_by, delivery_notes)
        VALUES
        (:order_id, :customer_id, "PENDING", :rider_name, :rider_phone, :address,
         :fee, :assigned_by, :notes)'
    );

    $stmt->execute([
        'order_id' => $data['order_id'],
        'customer_id' => $data['customer_id'],
        'rider_name' => $data['rider_name'] ?? null,
        'rider_phone' => $data['rider_phone'] ?? null,
        'address' => $data['delivery_address'],
        'fee' => $data['delivery_fee'] ?? 0,
        'assigned_by' => $user['id'],
        'notes' => $data['delivery_notes'] ?? null,
    ]);

    $deliveryId = (int) $db->lastInsertId();

    if ((float) ($data['delivery_fee'] ?? 0) > 0) {
        $finance = $db->prepare(
            'INSERT INTO finance_transactions
            (transaction_type, category, amount, reference_number, notes, created_by)
            VALUES ("INCOME", "Delivery Income", :amount, :reference, :notes, :created_by)'
        );

        $finance->execute([
            'amount' => $data['delivery_fee'],
            'reference' => 'DELIVERY-FEE-' . $deliveryId,
            'notes' => 'Delivery fee recorded from delivery order',
            'created_by' => $user['id'],
        ]);
    }

    AuditLogger::log(
        (int) $user['id'],
        'DELIVERIES',
        'CREATE_DELIVERY_ORDER',
        $deliveryId,
        null,
        $data
    );

    Response::success([
        'delivery_id' => $deliveryId,
    ], 'Delivery order created');
}

public function riders(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);
```

```
    $stmt = Database::connection()->query(
        'SELECT * FROM riders ORDER BY rider_name ASC'
    );

    Response::success(['riders' => $stmt->fetchAll()]);
}

public function storeRider(): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        'INSERT INTO riders
        (rider_name, rider_phone, rider_status, notes)
        VALUES (:name, :phone, :status, :notes)'
    );

    $stmt->execute([
        'name' => $data['rider_name'],
        'phone' => $data['rider_phone'],
        'status' => $data['rider_status'] ?? 'ACTIVE',
        'notes' => $data['notes'] ?? null,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'DELIVERIES',
        'CREATE_RIDER',
        null,
        null,
        $data
    );

    Response::success([], 'Rider created');
}

public function assignRider(int $orderId): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        "UPDATE deliveries
         SET rider_name = :rider_name,
             rider_phone = :rider_phone,
             assigned_by = :assigned_by,
             delivery_status = 'PENDING'
         WHERE order_id = :order_id"
    );

    $stmt->execute([
        'rider_name' => $data['rider_name'],
        'rider_phone' => $data['rider_phone'],
        'assigned_by' => $user['id'],
        'order_id' => $orderId,
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'DELIVERIES',
        'ASSIGN_RIDER',
        $orderId,
        null,
        $data
    );

    Response::success([], 'Rider assigned');
}
```

Replace the existing `dispatch()` method with:

```
public function dispatch(int $orderId): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        "UPDATE deliveries
         SET delivery_status = 'DISPATCHED',
             rider_name = COALESCE(:rider_name, rider_name),
             rider_phone = COALESCE(:rider_phone, rider_phone),
             dispatched_at = NOW(),
             assigned_by = :assigned_by
         WHERE order_id = :order_id"
    );
```

```
    $stmt->execute([
        'order_id' => $orderId,
        'rider_name' => $data['rider_name'] ?? null,
        'rider_phone' => $data['rider_phone'] ?? null,
        'assigned_by' => $user['id'],
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'DISPATCH',
        'DISPATCH_ORDER',
        $orderId,
        null,
        $data
    );

    Response::success([], 'Order dispatched');
}
```

Replace the existing `deliver()` method with:

```
public function deliver(int $orderId): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);

    $stmt = Database::connection()->prepare(
        "UPDATE deliveries
         SET delivery_status = 'DELIVERED',
             delivered_at = NOW()
         WHERE order_id = :order_id"
    );

    $stmt->execute(['order_id' => $orderId]);

    AuditLogger::log(
        (int) $user['id'],
        'DISPATCH',
        'MARK_DELIVERED',
        $orderId
    );

    Response::success([], 'Order marked as delivered');
}
```

Add:

```
public function failDelivery(int $orderId): void
{
    $user = Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);
    $data = Request::body();

    $stmt = Database::connection()->prepare(
        "UPDATE deliveries
         SET delivery_status = 'FAILED',
             failed_at = NOW(),
             failed_reason = :reason
         WHERE order_id = :order_id"
    );

    $stmt->execute([
        'order_id' => $orderId,
        'reason' => $data['failed_reason'] ?? 'No reason provided',
    ]);

    AuditLogger::log(
        (int) $user['id'],
        'DISPATCH',
        'MARK_DELIVERY_FAILED',
        $orderId,
        null,
        $data
    );

    Response::success([], 'Delivery marked as failed');
}

public function performanceReport(): void
{
    Auth::requireRole(['SUPER_ADMIN', 'ADMIN', 'MANAGER', 'DISPATCHER']);

    $stmt = Database::connection()->query(
        "SELECT
            rider_name,
            rider_phone,
            COUNT(*) AS total_deliveries,
            SUM(CASE WHEN delivery_status = 'DELIVERED' THEN 1 ELSE 0 END) AS delivered_count,
            SUM(CASE WHEN delivery_status = 'FAILED' THEN 1 ELSE 0 END) AS failed_count,
```

```
            COALESCE(SUM(delivery_fee), 0) AS delivery_fee_total
        FROM deliveries
        GROUP BY rider_name, rider_phone
        ORDER BY delivered_count DESC, failed_count ASC"
    );

    Response::success(['delivery_performance' => $stmt->fetchAll()]);
}
```

19F. Update

apps/admin-web/src/features/operations/api/operationsApi.js

Add methods:

```
createDeliveryOrder: async (payload) => {
  const { data } = await apiClient.post('/deliveries', payload)
  return data
},

riders: async () => {
  const { data } = await apiClient.get('/riders')
  return data.data.riders
},

createRider: async (payload) => {
  const { data } = await apiClient.post('/riders', payload)
  return data
},

assignRider: async (orderId, payload) => {
  const { data } = await apiClient.post(`/dispatch/orders/${orderId}/assign-rider`, payload)
  return data
},

failDelivery: async (orderId, payload) => {
  const { data } = await apiClient.post(`/dispatch/orders/${orderId}/fail`, payload)
  return data
},

deliveryPerformanceReport: async () => {
  const { data } = await apiClient.get('/dispatch/performance-report')
  return data.data.delivery_performance
},
```

19G.

apps/admin-web/src/features/deliveries/pages/DeliveryOrdersPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function DeliveryOrdersPage() {
  return (
    <OperationalFormPage
      title="Delivery Orders"
      subtitle="Create and monitor delivery orders"
      queryKey="delivery-orders"
      queryFn={operationsApi.dispatchOrders}
      mutationFn={operationsApi.createDeliveryOrder}
      initialForm={{
        order_id: '',
        customer_id: '',
        delivery_address: '',
        delivery_fee: '',
        rider_name: '',
        rider_phone: '',
        delivery_notes: '',
      }}
      buildPayload={(form) => ({
        ...form,
        order_id: Number(form.order_id),
        customer_id: Number(form.customer_id),
        delivery_fee: Number(form.delivery_fee || 0),
      })}
      fields={[
        { name: 'order_id', label: 'Order ID', type: 'number' },
        { name: 'customer_id', label: 'Customer ID', type: 'number' },
        { name: 'delivery_address', label: 'Delivery Address', type: 'textarea' },
        { name: 'delivery_fee', label: 'Delivery Fee', type: 'number' },
        { name: 'rider_name', label: 'Rider Name Optional' },
```

```
        { name: 'rider_phone', label: 'Rider Phone Optional' },
        { name: 'delivery_notes', label: 'Delivery Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'order_number', label: 'Order' },
        { key: 'customer_name', label: 'Customer' },
        { key: 'delivery_status', label: 'Status' },
        { key: 'delivery_fee', label: 'Fee' },
        { key: 'rider_name', label: 'Rider' },
        { key: 'rider_phone', label: 'Rider Phone' },
      ]}
      submitLabel="Create Delivery"
    />
  )
}
```

19H.

apps/admin-web/src/features/deliveries/pages/RidersPage.jsx

```
import React from 'react'
import OperationalFormPage from '../../../components/common/OperationalFormPage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function RidersPage() {
  return (
    <OperationalFormPage
      title="Riders"
      subtitle="Create and monitor delivery riders"
      queryKey="riders"
      queryFn={operationsApi.riders}
      mutationFn={operationsApi.createRider}
      initialForm={{
        rider_name: '',
        rider_phone: '',
        rider_status: 'ACTIVE',
        notes: '',
      }}
      buildPayload={(form) => form}
      fields={[
        { name: 'rider_name', label: 'Rider Name' },
        { name: 'rider_phone', label: 'Rider Phone' },
        {
          name: 'rider_status',
          label: 'Rider Status',
          type: 'select',
          options: [
            { value: 'ACTIVE', label: 'Active' },
            { value: 'INACTIVE', label: 'Inactive' },
            { value: 'SUSPENDED', label: 'Suspended' },
          ],
        },
        { name: 'notes', label: 'Notes', type: 'textarea' },
      ]}
      columns={[
        { key: 'rider_name', label: 'Rider' },
        { key: 'rider_phone', label: 'Phone' },
        { key: 'rider_status', label: 'Status' },
        { key: 'notes', label: 'Notes' },
      ]}
      submitLabel="Create Rider"
    />
  )
}
```

19I.

apps/admin-web/src/features/deliveries/pages/DispatchControlPage.jsx

```
import React, { useState } from 'react'
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import Button from '../../../components/ui/Button'
import Input from '../../../components/ui/Input'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function DispatchControlPage() {
  const qc = useQueryClient()
```

```
  const [assignForm, setAssignForm] = useState({
    order_id: '',
    rider_name: '',
    rider_phone: '',
  })

  const [failForm, setFailForm] = useState({
    order_id: '',
    failed_reason: '',
  })

  const { data, isLoading, error } = useQuery({
    queryKey: ['dispatch-control'],
    queryFn: operationsApi.dispatchOrders,
  })

  const assignMutation = useMutation({
    mutationFn: (payload) =>
      operationsApi.assignRider(payload.order_id, {
        rider_name: payload.rider_name,
        rider_phone: payload.rider_phone,
      }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['dispatch-control'] }),
  })

  const dispatchMutation = useMutation({
    mutationFn: (orderId) => operationsApi.dispatchOrder(orderId, {}),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['dispatch-control'] }),
  })

  const deliverMutation = useMutation({
    mutationFn: operationsApi.markDelivered,
    onSuccess: () => qc.invalidateQueries({ queryKey: ['dispatch-control'] }),
  })

  const failMutation = useMutation({
    mutationFn: (payload) =>
      operationsApi.failDelivery(payload.order_id, {
        failed_reason: payload.failed_reason,
      }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['dispatch-control'] }),
  })

  const rows = (data || []).map((item) => ({
    ...item,
    actions: (
      <div style={{ display: 'flex', gap: 8, flexWrap: 'wrap' }}>
        <Button onClick={() => dispatchMutation.mutate(item.order_id)}>Dispatch</Button>
        <Button onClick={() => deliverMutation.mutate(item.order_id)}>Delivered</Button>
      </div>
    ),
  }))

  return (
    <Page title="Dispatch Control" subtitle="Assign riders, dispatch, deliver, or fail delivery">
      <Card title="Assign Rider">
        <form
          onSubmit={(e) => {
            e.preventDefault()
            assignMutation.mutate(assignForm)
          }}
          style={{ display: 'grid', gap: 10, maxWidth: 560 }}
        >
          <Input placeholder="Order ID" value={assignForm.order_id} onChange={(e) => setAssignForm({
...assignForm, order_id: e.target.value })} />
          <Input placeholder="Rider Name" value={assignForm.rider_name} onChange={(e) => setAssignForm({
...assignForm, rider_name: e.target.value })} />
          <Input placeholder="Rider Phone" value={assignForm.rider_phone} onChange={(e) => setAssignForm({
...assignForm, rider_phone: e.target.value })} />
          <Button type="submit">Assign Rider</Button>
        </form>
      </Card>

      <Card title="Mark Delivery Failed">
        <form
          onSubmit={(e) => {
            e.preventDefault()
            failMutation.mutate(failForm)
          }}
          style={{ display: 'grid', gap: 10, maxWidth: 560 }}
        >
          <Input placeholder="Order ID" value={failForm.order_id} onChange={(e) => setFailForm({ ...failForm,
order_id: e.target.value })} />
          <Input placeholder="Failed Reason" value={failForm.failed_reason} onChange={(e) => setFailForm({
...failForm, failed_reason: e.target.value })} />
          <Button type="submit">Mark Failed</Button>
```

```
        </form>
      </Card>

      <Card title="Delivery Queue">
        {isLoading ? <LoadingBlock label="Loading deliveries..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}
        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'order_number', label: 'Order' },
              { key: 'customer_name', label: 'Customer' },
              { key: 'delivery_status', label: 'Status' },
              { key: 'rider_name', label: 'Rider' },
              { key: 'rider_phone', label: 'Phone' },
              { key: 'actions', label: 'Actions' },
            ]}
            rows={rows}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

19J.

apps/admin-web/src/features/deliveries/pages/DeliveryPerformanceReportPage.jsx

```
import React from 'react'
import { useQuery } from '@tanstack/react-query'
import Page from '../../../components/ui/Page'
import Card from '../../../components/ui/Card'
import DataTable from '../../../components/ui/DataTable'
import LoadingBlock from '../../../components/ui/LoadingBlock'
import StatusMessage from '../../../components/ui/StatusMessage'
import { operationsApi } from '../../operations/api/operationsApi'

export default function DeliveryPerformanceReportPage() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['delivery-performance-report'],
    queryFn: operationsApi.deliveryPerformanceReport,
  })

  return (
    <Page title="Delivery Performance" subtitle="Rider delivery success, failure, and fee report">
      <Card>
        {isLoading ? <LoadingBlock label="Loading delivery performance..." /> : null}
        {error ? <StatusMessage error={error.message} /> : null}

        {!isLoading && !error ? (
          <DataTable
            columns={[
              { key: 'rider_name', label: 'Rider' },
              { key: 'rider_phone', label: 'Phone' },
              { key: 'total_deliveries', label: 'Total' },
              { key: 'delivered_count', label: 'Delivered' },
              { key: 'failed_count', label: 'Failed' },
              { key: 'delivery_fee_total', label: 'Delivery Fees' },
            ]}
            rows={data || []}
          />
        ) : null}
      </Card>
    </Page>
  )
}
```

19K. Update

apps/admin-web/src/app/routes.jsx

Add imports:

```
import DeliveryOrdersPage from '../features/deliveries/pages/DeliveryOrdersPage'
import RidersPage from '../features/deliveries/pages/RidersPage'
import DispatchControlPage from '../features/deliveries/pages/DispatchControlPage'
import DeliveryPerformanceReportPage from '../features/deliveries/pages/DeliveryPerformanceReportPage'
```

Add routes:

```
<Route path="/deliveries/orders" element={<DeliveryOrdersPage />} />
<Route path="/deliveries/riders" element={<RidersPage />} />
```

```
<Route path="/deliveries/dispatch-control" element={<DispatchControlPage />} />
<Route path="/deliveries/performance" element={<DeliveryPerformanceReportPage />} />
```

19L. Update

apps/admin-web/src/components/common/SidebarNav.jsx

Add links:

```
['Delivery Orders', '/deliveries/orders'],
['Riders', '/deliveries/riders'],
['Dispatch Control', '/deliveries/dispatch-control'],
['Delivery Performance', '/deliveries/performance'],
```

19M.

docs/DELIVERY_DISPATCH_DEEPENING_PACK.md

```
# Batch 19 — Delivery + Dispatch Deepening Pack

## System

Meat Lovers CIMS powered by YohPal

## Purpose

This batch deepens the delivery and dispatch system from simple delivery status tracking into full delivery
control.

## Features Added

1. Delivery order creation
2. Rider creation
3. Rider assignment
4. Dispatch status updates
5. Mark delivered action
6. Failed delivery reason capture
7. Delivery fee tracking
8. Delivery performance report

## Delivery Status Flow

```text
PENDING → DISPATCHED → DELIVERED
```

Alternative failed flow:

```
PENDING → DISPATCHED → FAILED
```

Delivery Fee Tracking

Delivery fee is recorded into:

```
finance_transactions
```

Category:

```
Delivery Income
```

Rider Performance

The delivery performance report tracks:

- total deliveries
- delivered count
- failed count
- delivery fee total

Business Control Value

This helps Meat Lovers:

- track who handled delivery
- know which rider was assigned
- monitor failed deliveries
- record delivery income
- compare rider performance
```

- reduce delivery leakage

Batch 19 Outcome

Delivery + dispatch now includes:

✅ delivery order creation
✅ rider assignment
✅ dispatch status updates
✅ failed delivery reason capture
✅ delivery fee tracking
✅ delivery performance reporting

Next Smart Move

Build **Batch 20 — Supplier + Storekeeping Deepening Pack**, including supplier invoices, receiving notes, purchase approval, stock receiving, stock transfer from store to kitchen/bar, stock movement report, and supplier performance report.